

Docker Fundamentals

1. What is Docker?

Docker is a containerization platform that packages an application together with its dependencies into a portable unit called a container. Containers are lightweight, isolated process environments that share the host operating system kernel. Docker helps developers build, test, ship, and run applications consistently across different environments.

2. Docker Architecture

Docker follows a client-server architecture. The Docker client, such as the docker command-line tool, sends requests to the Docker daemon. The Docker daemon builds images, creates and manages containers, networks, and volumes. Docker registries such as Docker Hub store and distribute images.

Basic flow: Docker CLI → Docker Daemon → Images / Containers / Networks / Volumes

3. Images vs Containers

Docker Image: A read-only template containing the application code, runtime, libraries, and configuration required to create a container.

Docker Container: A running or stopped instance created from an image. A container provides an isolated environment in which the application runs. A useful analogy is: an image is like a class or blueprint, while a container is an instance created from that blueprint.

4. Dockerfile

A Dockerfile is a text file containing instructions for building a Docker image. Common instructions include FROM for the base image, WORKDIR for the working directory, COPY for application files, RUN for build commands, EXPOSE for documenting ports, and CMD for the default command.

5. Docker Volumes

Docker volumes provide persistent storage for containers. Normally, data written inside a container can be lost when the container is removed. A volume stores data outside the container's writable

layer and can be reused by another container. Volumes are commonly used for databases such as MongoDB.

6. Docker Networks

Docker networks allow containers to communicate with each other and provide network isolation. On a user-defined bridge network, containers can communicate using service or container names. For example, in a Compose application, a backend can connect to MongoDB using a hostname such as `mongodb` instead of an IP address.

7. Docker Compose

Docker Compose is a tool for defining and managing multi-container applications using a YAML file, commonly named `compose.yaml` or `docker-compose.yml`. It can define services, images, builds, ports, networks, volumes, and environment variables. Commands such as `docker compose up` and `docker compose down` simplify application deployment and management.

8. Docker Hub

Docker Hub is a cloud-based registry for storing and sharing Docker images. Users can pull public images, such as official database images, and push their own images for distribution. A typical image name follows the format `username/repository:tag`, for example `username/my-app:latest`.